

Лабораторная работа №2

Компьютерный практикум по статистическому анализу

Надежда Александровна Рогожина

2025-09-27

Содержание I

1. Информация

2. Введение

3. Выполнение лабораторной работы

4. Выводы

Раздел 1

1. Информация

1.1 Докладчик

► Рогожина Надежда Александровна

1.1 Докладчик

- ▶ Рогожина Надежда Александровна
- ▶ студентка 4 курса НФИбд-02-22

1.1 Докладчик

- ▶ Рогожина Надежда Александровна
- ▶ студентка 4 курса НФИбд-02-22
- ▶ Российский университет дружбы народов им. П. Лумумбы

1.1 Докладчик

- ▶ Рогожина Надежда Александровна
- ▶ студентка 4 курса НФИбд-02-22
- ▶ Российский университет дружбы народов им. П. Лумумбы
- ▶ 1132222840@rudn.ru

1.1 Докладчик

- ▶ Рогожина Надежда Александровна
- ▶ студентка 4 курса НФИбд-02-22
- ▶ Российский университет дружбы народов им. П. Лумумбы
- ▶ 1132222840@rudn.ru
- ▶ <https://MikoGreen.github.io/ru/>

Раздел 2

2. Введение

2.1 Цель работы

Основная цель работы — изучить несколько структур данных, реализованных в Julia, научиться применять их и операции над ними для решения задач.

2.2 Задание

1. Используя Jupyter Lab, повторите примеры из раздела 2.2.

2.2 Задание

1. Используя Jupyter Lab, повторите примеры из раздела 2.2.
2. Выполните задания для самостоятельной работы (раздел 2.4).

2.3 Теоретическое введение

Научные вычисления традиционно требуют высочайшей производительности, однако эксперты по доменам в значительной степени перенесены на более медленные динамические языки для ежедневной работы. К счастью, методы современного языка и компилятора позволяют в основном устранить компромисс производительности и обеспечивать достаточно продуктивную среду для прототипирования и достаточно эффективно для развертывания применений, интенсивных производительности. Язык программирования Julia заполняет эту роль: это гибкий динамический язык, подходящий для научных и численных вычислений, с производительностью, сравнимой с традиционными статичными языками.

Раздел 3

3. Выполнение лабораторной работы

3.1 Задание №1

Первым заданием было повторить примеры, где были показаны основы работы с кортежами, словарями, множествами и массивами.

3.2 Примеры

Рассмотрим несколько структур данных, реализованных в Julia. Несколько функций (методов), общих для всех структур данных:

- `isempty()` — проверяет, пуста ли структура данных;
- `length()` — возвращает длину структуры данных;
- `in()` — проверяет принадлежность элемента к структуре;
- `unique()` — возвращает коллекцию уникальных элементов структуры;
- `reduce()` — свертывает структуру данных в соответствии с заданным бинарным оператором;
- `maximum()` (или `minimum()`) — возвращает наибольший (или наименьший) результат вызова функции для каждого элемента структуры данных.

```
In [1]: {}  
Out[1]: {}  
  
In [5]: favoritelang = ("Python", "Julia", "R")  
Out[5]: ("Python", "Julia", "R")  
  
In [3]: x1 = (1, 2, 3)  
Out[3]: (1, 2, 3)  
  
In [4]: x2 = (1, 2.0, "tmp")  
Out[4]: (1, 2.0, "tmp")  
  
In [6]: x3 = (a=2, b=1+2)  
Out[6]: (a = 2, b = 3)  
  
In [8]: length(x3)  
Out[8]: 2  
  
In [10]: x2[1], x2[2], x2[3]  
Out[10]: (1, 2.0, "tmp")  
  
In [11]: c = x1[2] + x1[3]  
Out[11]: 5  
  
In [12]: x3.a, x3.b, x3[2]  
Out[12]: (2, 3, 3)  
  
In [13]: in("tmp", x2), 0 in x2  
Out[13]: (true, false)
```

Рисунок 1: Кортежи

3.3 Примеры

Словари

```
In [28]: phonebook = Dict{"Иванов И.И." => ("867-5309", "333-5544"), "Бузыгалтея" => "555-2368"}
```

```
Out[28]: Dict{String, Any} with 2 entries:  
"Бузыгалтея" => "555-2368"  
"Иванов И.И." => ("867-5309", "333-5544")
```

```
In [29]: keys(phonebook)
```

```
Out[29]: KeySet for a Dict{String, Any} with 2 entries. Keys:  
"Бузыгалтея"  
"Иванов И.И."
```

```
In [30]: values(phonebook)
```

```
Out[30]: ValueIterator for a Dict{String, Any} with 2 entries. Values:  
"555-2368"  
("867-5309", "333-5544")
```

```
In [31]: pairs(phonebook)
```

```
Out[31]: Dict{String, Any} with 2 entries:  
"Бузыгалтея" => "555-2368"  
"Иванов И.И." => ("867-5309", "333-5544")
```

```
In [32]: haskey(phonebook, "Иванов И.И.")
```

```
Out[32]: true
```

```
In [33]: phonebook["Сидоров П.С."] = "555-3344"  
pairs(phonebook)
```

```
Out[33]: Dict{String, Any} with 3 entries:  
"Сидоров П.С." => "555-3344"  
"Бузыгалтея" => "555-2368"  
"Иванов И.И." => ("867-5309", "333-5544")
```

```
In [34]: pop!(phonebook, "Иванов И.И.")  
pairs(phonebook)
```

```
Out[34]: Dict{String, Any} with 2 entries:  
"Сидоров П.С." => "555-3344"  
"Бузыгалтея" => "555-2368"
```

Рисунок 2: Словари

3.4 Примеры

```
In [35]: a = Dict("foo" => 0.0, "bar" => 42.0);  
        b = Dict("baz" => 17, "bar" => 13.0);  
        merge(a,b)
```

```
Out[35]: Dict{String, Real} with 3 entries:  
  "bar" => 13.0  
  "baz" => 17  
  "foo" => 0.0
```

```
In [36]: a = Dict("foo" => 0.0, "bar" => 42.0);  
        b = Dict("baz" => 17, "bar" => 13.0);  
        merge(b,a)
```

```
Out[36]: Dict{String, Real} with 3 entries:  
  "bar" => 42.0  
  "baz" => 17  
  "foo" => 0.0
```

Рисунок 3: Словари

3.5 Примеры

Множества

```
In [49]: A = Set([1, 3, 4, 5])
```

```
Out[49]: Set{Int64} with 4 elements:  
5  
4  
3  
1
```

```
In [50]: B = Set("abracadabra")
```

```
Out[50]: Set{Char} with 5 elements:  
'a'  
'd'  
'r'  
'k'  
'b'
```

```
In [51]: S1 = Set([1,2]);  
S2 = Set([3,4]);  
issetequal(S1,S2)
```

```
Out[51]: false
```

```
In [52]: S3 = Set([1,2,2,3,1,2,3,2,1]);  
S4 = Set([2,3,1]);  
issetequal(S3,S4)
```

```
Out[52]: true
```

Рисунок 4: Множества

3.6 Примеры

```
In [53]: C=union(S1,S2)
Out[53]: Set{Int64} with 4 elements:
         4
         2
         3
         1

In [54]: D = intersect(S1,S3)
Out[54]: Set{Int64} with 2 elements:
         2
         1

In [58]: E = setdiff(S4,S2)
Out[58]: Set{Int64} with 2 elements:
         2
         1

In [55]: issubset(S1,S4)
Out[55]: true

In [59]: push!(S4, 99)
Out[59]: Set{Int64} with 4 elements:
         2
        99
         3
         1

In [60]: pop!(S4)
Out[60]: 2

In [61]: S4
Out[61]: Set{Int64} with 3 elements:
        99
         3
         1
```

Рисунок 5: Множества

3.7 Примеры

Массивы

```
In [65]: empty_array_3 = {Float64}[]
Out[65]: Float64[]

In [66]: a = [1, 2, 3]
Out[66]: 3-element Vector{Int64}:
 1
 2
 3

In [67]: b = [1 2 3]
Out[67]: 1×3 Matrix{Int64}:
 1 2 3

In [68]: A = [[1, 2, 3] [4, 5, 6] [7, 8, 9]]
Out[68]: 3×3 Matrix{Int64}:
 1 4 7
 2 5 8
 3 6 9

In [70]: B = [[1 2 3]; [4 5 6]; [7 8 9]]
Out[70]: 3×3 Matrix{Int64}:
 1 2 3
 4 5 6
 7 8 9

In [71]: c = rand(1,8)
Out[71]: 1×8 Matrix{Float64}:
 0.363351  0.438181  0.976741  0.717742  - 0.538385  0.84316  0.683375

In [72]: C = rand(2,3)
Out[72]: 2×3 Matrix{Float64}:
 0.543983  0.116562  0.267335
 0.88741  0.8731718  0.485564

In [76]: D = rand(4, 3, 2)
Out[76]: 4×3×2 Array{Float64, 3}:
[:, :, 1] =
 0.5168189  0.705842  0.575112
 0.485165  0.20176  0.824653
 0.298807  0.046161  0.244776
 0.674848  0.759963  0.96651

[:, :, 2] =
 0.273296  0.433723  0.112767
 0.382967  0.385296  0.0423815
 0.381404  0.324486  0.98377
 0.387533  0.163487  0.495973
```

Рисунок 6: Массивы

3.8 Примеры

```
In [75]: roots = [sqrt(i) for i in 1:10]
Out[75]: 10-element Vector{Float64}:
 1.0
 1.4142135623730951
 1.7320508075688772
 2.0
 2.23606797749979
 2.449489742783178
 2.6457513110645907
 2.8284271247461903
 3.0
 3.1622776601683795

In [76]: ar_1 = [3*i^2 for i in 1:2:9]
Out[76]: 5-element Vector{Int64}:
 3
 27
 75
 147
 243

In [77]: ar_2=[i^2 for i=1:10 if (i^2%5!=0 && i^2%4!=0)]
Out[77]: 4-element Vector{Int64}:
 1
 9
 49
 81
```

Рисунок 7: Массивы

3.9 Примеры

```
In [78]: ones(5)
Out[78]: 5-element Vector{Float64}:
 1.0
 1.0
 1.0
 1.0
 1.0

In [79]: ones(2,3)
Out[79]: 2×3 Matrix{Float64}:
 1.0  1.0  1.0
 1.0  1.0  1.0

In [80]: zeros(4)
Out[80]: 4-element Vector{Float64}:
 0.0
 0.0
 0.0
 0.0

In [86]: fill(3.5,(3,2))
Out[86]: 3×2 Matrix{Float64}:
 3.5  3.5
 3.5  3.5
 3.5  3.5

In [89]: repeat([1,2],3,3)
Out[89]: 6×3 Matrix{Int64}:
 1  1  1
 2  2  2
 1  1  1
 2  2  2
 1  1  1
 2  2  2

In [88]: repeat([1 2],3,3)
Out[88]: 3×6 Matrix{Int64}:
 1  2  1  2  1  2
 1  2  1  2  1  2
 1  2  1  2  1  2
```

3.10 Примеры

```
In [90]: a = collect(1:12)
         b = reshape(a,(2,6))

Out[90]: 2×6 Matrix{Int64}:
 1  3  5  7  9 11
 2  4  6  8 10 12

In [93]: b'

Out[93]: 6×2 adjoint(::Matrix{Int64}) with eltype Int64:
 1  2
 3  4
 5  6
 7  8
 9 10
11 12

In [95]: c = transpose(b)

Out[95]: 6×2 transpose(::Matrix{Int64}) with eltype Int64:
 1  2
 3  4
 5  6
 7  8
 9 10
11 12
```

Рисунок 9: Массивы

3.11 Примеры

```
In [96]: ar = rand(10:20, 10, 5)
```

```
Out[96]: 10x5 Matrix{Int64}:  
 15 19 18 13 16  
 10 11 18 10 15  
 19 12 19 18 15  
 15 12 16 13 16  
 13 17 18 14 20  
 10 20 20 18 10  
 18 10 15 12 10  
 13 20 17 16 13  
 17 16 19 16 11  
 11 11 10 14 16
```

```
In [100]: ar[:, 2]
```

```
Out[100]: 10-element Vector{Int64}:  
 19  
 11  
 12  
 12  
 17  
 20  
 10  
 20  
 16  
 11
```

```
In [98]: ar[:, [2, 5]]
```

```
Out[98]: 10x2 Matrix{Int64}:  
 19 16  
 11 15  
 12 15  
 12 16  
 17 20  
 20 10  
 10 10  
 20 13  
 16 11  
 11 16
```

Рисунок 10: Массивы

3.12 Примеры

```
In [101]: ar[:, 2:4]
Out[101]: 10x3 Matrix{Int64}:
          19 18 13
          11 18 10
          12 19 18
          12 16 13
          17 18 14
          20 20 18
          10 15 12
          20 17 16
          16 19 16
          11 10 14

In [102]: ar[[2, 4, 6], [1, 5]]
Out[102]: 3x2 Matrix{Int64}:
          10 15
          15 16
          10 10

In [103]: ar[1, 3:end]
Out[103]: 3-element Vector{Int64}:
          18
          13
          16
```

Рисунок 11: Массивы

3.13 Примеры

```
In [104]: sort(ar,dims=1)
```

```
Out[104]: 10x5 Matrix(Int64):  
10 10 10 10 10  
10 11 15 12 10  
11 11 16 13 11  
13 12 17 13 13  
13 12 18 14 15  
15 16 18 14 15  
15 17 18 16 16  
17 19 19 16 16  
18 20 19 18 16  
19 20 20 18 20
```

```
In [105]: sort(ar,dims=2)
```

```
Out[105]: 10x5 Matrix(Int64):  
13 15 16 18 19  
10 10 11 15 18  
12 15 18 19 19  
12 13 15 16 16  
13 14 17 18 20  
10 10 18 20 20  
10 10 12 15 18  
13 13 16 17 20  
11 16 16 17 19  
10 11 11 14 16
```

Рисунок 12: Массивы

[illegible]

Рисунок 13: Массивы

3.15 Задание №2

Далее, необходимо было выполнить *Задания для самостоятельного выполнения*

3.16 Задачи для самостоятельного выполнения

1. Операции над множествами

```
In [110]: A = Set([0, 3, 4, 9])  
          B = Set([1, 3, 4, 7])  
          C = Set([0, 1, 2, 4, 7, 8, 9])  
  
          P = union(intersect(A, B), intersect(A, B), intersect(A, C), intersect(B, C))  
  
Out[110]: Set(Int64) with 6 elements:  
          0  
          4  
          7  
          9  
          3  
          1
```

Рисунок 14: Объединение множеств

3.17 Задачи для самостоятельного выполнения

2. Собственные примеры

```
In [133]: S1 = Set(["apple", 42, 3.14])  
          S2 = Set([42, "banana", 2.75])
```

```
Out[133]: Set[Any] with 3 elements:  
          2.75  
          "banana"  
          42
```

```
In [134]: union(S1, S2)
```

```
Out[134]: Set[Any] with 5 elements:  
          2.75  
          3.14  
          42  
          "banana"  
          "apple"
```

```
In [135]: intersect(S1, S2)
```

```
Out[135]: Set[Any] with 1 element:  
          42
```

```
In [136]: setdiff(S1, S2)
```

```
Out[136]: Set[Any] with 2 elements:  
          3.14  
          "apple"
```

```
In [137]: push!(S1, 125)
```

```
Out[137]: Set[Any] with 4 elements:  
          3.14  
          42  
          "apple"  
          125
```

```
In [138]: pop!(S2)
```

```
Out[138]: 2.75
```

Рисунок 15: Собственные примеры

3.18 Задачи для самостоятельного выполнения

3. Работа с массивами
 18. Для удобства вывода информации - будем транскодировать массивы, где это возможно!

```

In [138]: N=24
Out[138]: 24

In [138]: a = transpose([1 for i in 1:N])
Out[138]: 1x24 transpose{Vector{Int64}} with eltype Int64:
 1 2 3 4 5 6 7 8 9 10 11 12 ... 20 21 22 23 24

In [137]: b = transpose[0:1:24]
Out[137]: 1x24 transpose{Vector{Int64}} with eltype Int64:
 24 23 22 21 20 19 18 17 16 15 ... 12 10 9 8 7 6 5 4 3 2 1

In [139]: transpose{cvt[i] for i in 1:N/3}, end{end(N)-1:3:1}
Out[139]: 1x47 transpose{Vector{Int64}} with eltype Int64:
 5 3 3 4 5 6 7 8 9 10 11 12 ... 31 30 9 8 7 6 5 4 3 2 1

In [139]: tmp = [4,5,3]
Out[139]: 3-element Vector{Int64}:
 4
 5
 3

In [141]: transpose{fill{tmp[i], (N, 1)}}
Out[141]: 1x30 transpose{Matrix{Int64}} with eltype Int64:
 4 4 4 4 4 4 4 4 4 4

In [140]: transpose{repeat{tmp, inner=50}}
Out[140]: 1x30 transpose{Vector{Int64}} with eltype Int64:
 4 4 4 4 4 4 4 4 4 4 6 6 6 ... 6 6 3 3 3 3 3 3 3 3

In [144]: transpose{cvt[fill{tmp[i], (1, 1)}, fill{tmp[i], (N, 1)}, fill{tmp[i], (N, 1)}}
Out[144]: 1x1 transpose{Matrix{Int64}} with eltype Int64:
 4 4 4 4 4 4 4 4 4 4 4 6 6 ... 6 6 3 3 3 3 3 3 3 3

In [145]: transpose{cvt[fill{tmp[i], (N, 1)}, fill{tmp[i], (N, 1)}, fill{tmp[i], (N, 1)}]}
Out[145]: 1x60 transpose{Matrix{Int64}} with eltype Int64:
 4 4 4 4 4 4 4 4 4 4 6 6 6 ... 3 3 3 3 3 3 3 3 3 3

In [138]: aT = transpose{cvt[fill{tmp[i], (1, 1)}, fill{tmp[i], (1, 1)}, fill{tmp[i], (1, 1)}]}
Out[138]: 1x6 transpose{Matrix{Int64}} with eltype Int64:
 10 04 5 6 0 5

In [139]: sum{aT} -> x = 8, aT
Out[139]: 8

```

Рисунок 16: Работа с массивами

3.19 Задачи для самостоятельного выполнения

```
In [172]: x = transpose(collect(3:0.1:6))  
  
Out[172]: 1×31 transpose(::Vector{Float64}) with eltype Float64:  
 3.0 3.1 3.2 3.3 3.4 3.5 3.6 3.7 ... 5.4 5.5 5.6 5.7 5.8 5.9 6.0  
  
In [174]: y = exp.(x) .* cos.(x)  
  
Out[174]: 1×31 Matrix{Float64}:  
 -19.8845 -22.1788 -24.4987 -26.7732 ... 249.468 292.487 338.564 387.36  
  
In [175]: using Statistics  
          mean(y)  
  
Out[175]: 53.11374594642971
```

Рисунок 17: Работа с массивами

3.20 Задачи для самостоятельного выполнения

```
In [177]: i = collect(3:3:36)
          j = collect(1:1:34)

Out[177]: 12-element Vector{Int64}:
           1
           4
           7
          10
          13
          16
          19
          22
          25
          28
          31
          34

In [185]: x = 0.1
          y = 0.2
          xy = reshape(vcat([x^i for i in 1:12], [y^j for j in 1:12]), (12, 2))

Out[185]: 12x2 Matrix{Float64}:
 0.001  0.2
 1.0e-6  0.0016
 1.0e-9  1.20e-5
 1.0e-12  1.026e-7
 1.0e-15  8.192e-10
 1.0e-18  6.5536e-12
 1.0e-21  5.24288e-14
 1.0e-24  4.1943e-16
 1.0e-27  3.3554e-18
 1.0e-30  2.68435e-20
 1.0e-33  2.14748e-22
 1.0e-36  1.71799e-24
```

Рисунок 18: Работа с массивами

3.21 Задачи для самостоятельного выполнения

```
In [195]: M = 25
          [2**i/i for i in 1:M]

Out[195]: 25-element Vector{Float64}:
           2.0
           2.0
          2.6666666666666665
           4.0
           6.4
          10.666666666666666
          18.285714285714285
           32.0
          56.888888888888886
          102.4
          186.1818181818182
          341.3333333333333
          630.1538461538462
          1170.2857142857142
          2184.5333333333333
          4096.0
          7710.117647058823
          14503.555555555555
          27594.105263157893
          52428.8
          99864.30895238095
          190650.18181818182
          364722.0809505217
          699050.6666666666
           1.34217728e6
```

```
In [197]: N = 30
          [string("fn", string(i)) for i in 1:N]

Out[197]: 30-element Vector{String}:
          "fn1"
          "fn2"
          "fn3"
          "fn4"
          "fn5"
          "fn6"
          "fn7"
          "fn8"
          "fn9"
          "fn10"
          "fn11"
          "fn12"
          "fn13"
          ⋮
          "fn19"
          "fn20"
          "fn21"
          "fn22"
          "fn23"
          "fn24"
          "fn25"
          "fn26"
          "fn27"
          "fn28"
          "fn29"
          "fn30"
```

3.22 Задачи для самостоятельного выполнения

```
In [195]: n = 250
          x = rand(0:999, n)
          y = rand(0:999, n)

Out[195]: 250-element Vector{Int64}:
 387
 787
  38
 773
 568
 382
 364
 745
 385
  34
 971
 248
 539
  1
 634
 264
 600
 938
 879
 431
 521
 687
 371
  52
 987
  1

In [207]: transpose([2*(i-1) * i[i] for i in 1:n-1])
Out[207]: 1×288 transpose{::Vector{Int64}} with eltype Int64:
 278 -518 436 -382 -819 27 -182 - -285 -241 360 -120 577 -752

In [208]: transpose([2*(i+1) - x[i+2] for i in 1:n-2])
Out[208]: 1×288 transpose{::Vector{Int64}} with eltype Int64:
 1321 184 1386 1385 802 1308 1787 - 1240 2072 779 -37 248 1385

In [210]: transpose([sin(y[i]) * cos(x[i+1]) for i in 1:n-1])
Out[210]: 1×240 transpose{::Vector{Float64}} with eltype Float64:
 -0.080464 0.067388 1.8127 -1.18253 - -12.0952 1.77718 -0.515349

In [211]: transpose([cos(x[i-1]) / (x[i]-10) for i in 1:n-1])
Out[211]: 1×288 transpose{::Vector{Float64}} with eltype Float64:
 1.38055e-240 2.18397e-159 0.0 0.0 - 4.35218e-183 0.0 3.12444e-234
```

Рисунок 20: Работа с массивами

3.23 Задачи для самостоятельного выполнения

```

In [217]: [[i, y[i]] for i in range(100) if y[i] > 600]
Out[217]: 84-element Vector[Vector{Int64}]:
 [2, 787]
 [4, 775]
 [6, 745]
 [11, 973]
 [18, 856]
 [20, 828]
 [23, 999]
 [24, 888]
 [25, 888]
 [33, 894]
 [34, 884]
 [38, 858]
 [45, 827]
 |
 [223, 889]
 [224, 980]
 [225, 939]
 [227, 816]
 [253, 785]
 [258, 748]
 [259, 816]
 [243, 689]
 [243, 939]
 [243, 879]
 [248, 897]
 [248, 827]

In [228]: [x[i] for i in range(100) if y[i] > 600]
Out[228]: 84-element Vector{Int64}:
 546
 878
 982
 778
 288
 522
 538
 344
 489
 756
 489
 249
 86
 |
 583
 692
 538
 264
 372
 796
 887
 31
 512
 804
 31
 753

```

Рисунок 21: Работа с массивами

3.24 Задачи для самостоятельного выполнения

```

In [22]: x_mean = mean(x)
Out[22]: 497.776

In [22]: [abs(x[i] - x_mean)*0.5 for i in 1:n]
Out[22]: 250-element Vector{Float64}:
 3.1265959513885
 6.444558222503805
11.78012574228548
19.2318757757886
28.420461783907343
12.62974032557573
28.71772188246585
17.4408222389417
18.8912837572862
19.1083462332287
16.75977464318835
19.84378994858985
13.3803346454132
19.7287887548832
4.233128288272871
22.853880782533643
9.71451862223812
19.13657988823815
17.58478613478638
28.98151651534838
22.853880782533643
17.798282154397787
9.808880716748
18.97175195162426
5.784828378513855

In [23]: y_max = maximum(y)
Out[23]: 890

In [23]: count(abs(y[i] - y_max) <= 200 for i in 1:n)
Out[23]: 45

In [23]: count(x[i] >= x[i] * 2 == 0, x)
Out[23]: 132

In [23]: count(x[i] >= x[i] * 2 == 5, x)
Out[23]: 118

In [23]: count(x[i] >= x[i] * 7 == 0, x)
Out[23]: 87

```

Рисунок 22: Работа с массивами

3.25 Задачи для самостоятельного выполнения

```
In [231]: sort_ind = sortperm(y)
          x[sort_ind]

Out[231]: 258-element Vector{Int64}:
 717
 38
 531
 983
 661
 795
 734
 863
 359
 269
 694
 828
 385
  :
 901
 223
 277
 550
 344
 779
 694
 276
 753
 833
 382
 539

In [232]: y[sort_ind]

Out[232]: 258-element Vector{Int64}:
 0
 1
 1
 3
 6
 17
 18
 24
 30
 30
 33
 34
 35
  :
 949
 955
 957
 965
 968
 971
 978
 978
 987
 993
 996
 999
```

3.26 Задачи для самостоятельного выполнения

```
In [233]: sort(x, rev=true)[1:10]
Out[233]: 10-element Vector{Int64}:
 989
 984
 983
 974
 968
 963
 959
 947
 942
 938

In [234]: unique(x)
Out[234]: 226-element Vector{Int64}:
 488
 546
 359
 878
 915
 337
 927
 802
 744
 803
 779
 184
 326
 1
 796
 115
 379
 479
 11
 512
 864
 887
 181
 419
 751
 531

In [236]: squares = [1^2 for i in 1:100]
Out[236]: 100-element Vector{Int64}:
 1
 4
 9
 16
 25
 36
 49
 64
 81
 100
 121
 144
 169
```

Рисунок 24: Работа с массивами

3.27 Задачи для самостоятельного выполнения

```
In [240]: using Primes

In [244]: myprimes = primes(1000)

Out[244]: 100-element Vector{Int64}:
 2
 3
 5
 7
11
13
17
19
23
29
31
37
41
 53
 59
 67
 71
 73
 79
 83
 89
 97
101
103
107
109
113
127
131
137
139
149
151
157
163
167
173
179
181
187
191
193
197
201
211
223
227
229
233
239
241
251
257
263
269
271
277
281
283
293
299
307
311
313
317
331
337
347
349
353
359
367
373
379
383
389
397
401
409
419
421
431
433
439
443
449
457
461
463
467
473
479
487
491
499
503
509
517
521
523
529
533
541
547
557
563
569
571
577
587
593
599
601
607
613
617
619
623
629
631
637
641
643
647
653
659
661
667
671
673
677
683
687
691
697
701
709
713
719
727
733
739
743
751
757
761
769
773
779
787
797
809
811
821
823
827
829
833
839
847
853
857
859
863
869
877
881
883
887
893
899
907
911
913
917
919
923
929
931
937
941
943
947
953
959
967
971
973
977
983
989
991
993
997
1000

In [245]: myprimes[100]

Out[245]: 461

In [246]: myprimes[100:100]

Out[246]: 11-element Vector{Int64}:
 461
 463
 467
 473
 479
 487
 491
 499
 503
 509
 517
 521
 523

In [249]: (import Pkg; Pkg.add("Primes"))
```

Рисунок 25: Работа с массивами

3.28 Задачи для самостоятельного выполнения

```

In [247]: sum(2^i - 4*i^2 for i in 10:100)
Out[247]: 26952795

In [248]: N = 25
          sum((2^i-1)/i + (2^i-1)/(i-2) for i in 1:N)
Out[248]: 2.1295704368143902e9

In [249]: A = ones(10)
Out[249]: 10-element Vector{Float64}:
 0.0
 0.0
 0.0
 0.0
 0.0
 0.0
 0.0
 0.0
 0.0
 0.0

In [250]: A[1] = 2/3
Out[250]: 0.6666666666666666

In [254]: for i in 2:10
          A[i] = A[i-1] + (2^i)/(2^i + 1)
        end

In [256]: z = sum(A)
Out[256]: 6.37634613789762

```

Рисунок 26: Работа с массивами

Раздел 4

4. Выводы

4.1 Выводы

В ходе работы были изучены основы работы с разными типами структур в я.п. Julia.